UE23CS352B - Object Oriented Analysis & Design

Mini Project Report

Title: NexLearn-Learning Management System

Submitted by:

Team Members

NAME	SRN
Kathit Joshi	PES2UG23CS264
Kavyansh Jain	PES2UG23CS268
Kumarchandra Edupuganti	PES2UG23CS292
Laharish S	PES2UG23CS298

6th Semester-E

Faculty Name: Mr Nandhikesavan

January - May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)
100ft Ring Road, Bengaluru – 560 085, Karnataka, India

Problem Statement:

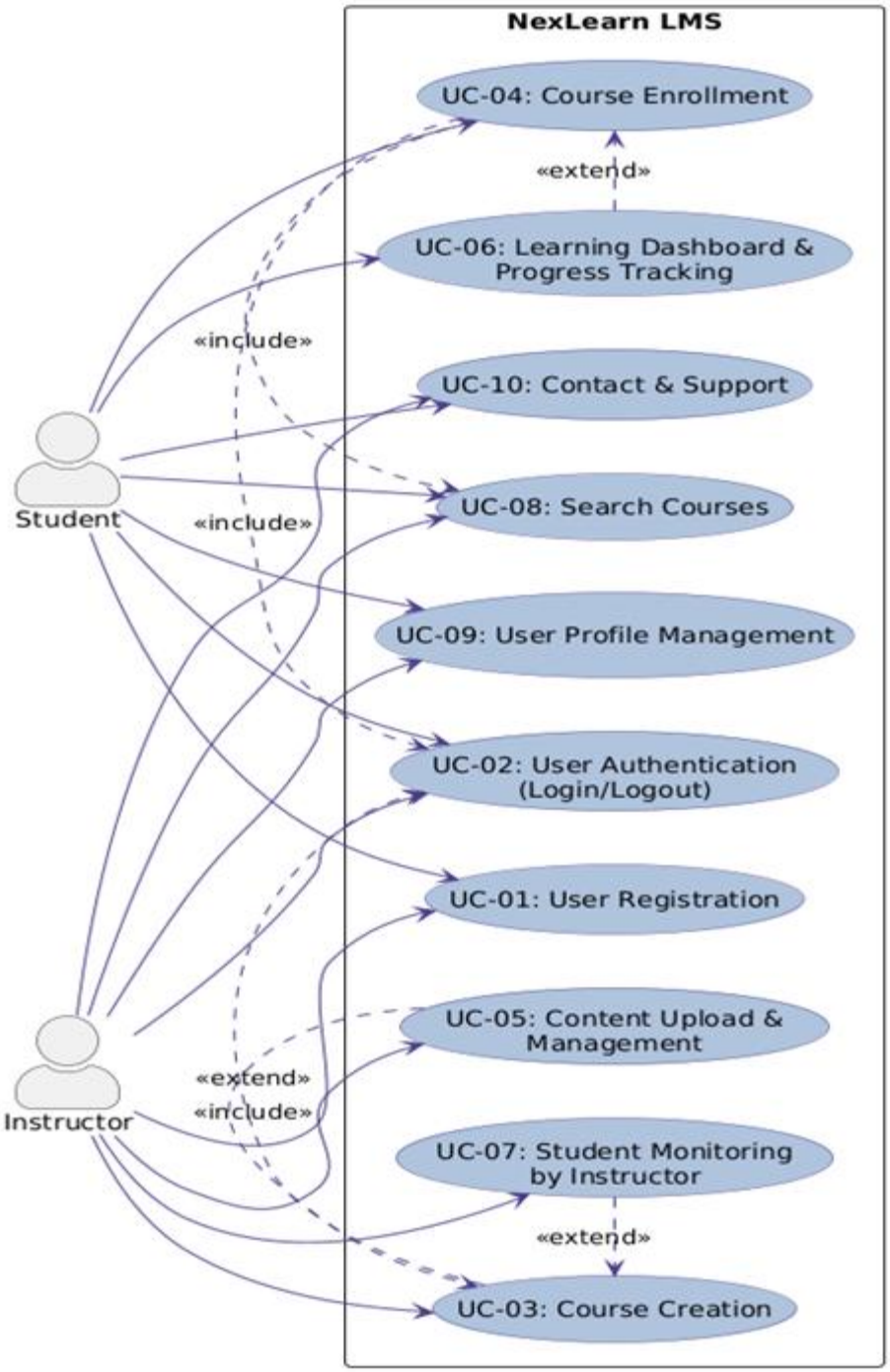
NexLearn is a web-based Learning Management System (LMS) developed to address the need for a structured, role-based platform where Instructors can create and manage educational content and Students can discover, enroll in, and track their learning progress. Traditional learning platforms are either too complex for academic use or lack the backend structure needed to demonstrate real-world software engineering principles. NexLearn solves this by providing a full-stack application built on Spring Boot 3.4 (MVC backend with JWT-based authentication), Next.js 15 (React frontend), and MongoDB Atlas (cloud NoSQL database) — demonstrating clean MVC architecture, SOLID principles, GRASP patterns, and GoF design patterns in a working, deployable system covering 10 distinct use cases across the Education domain.

Key Features:

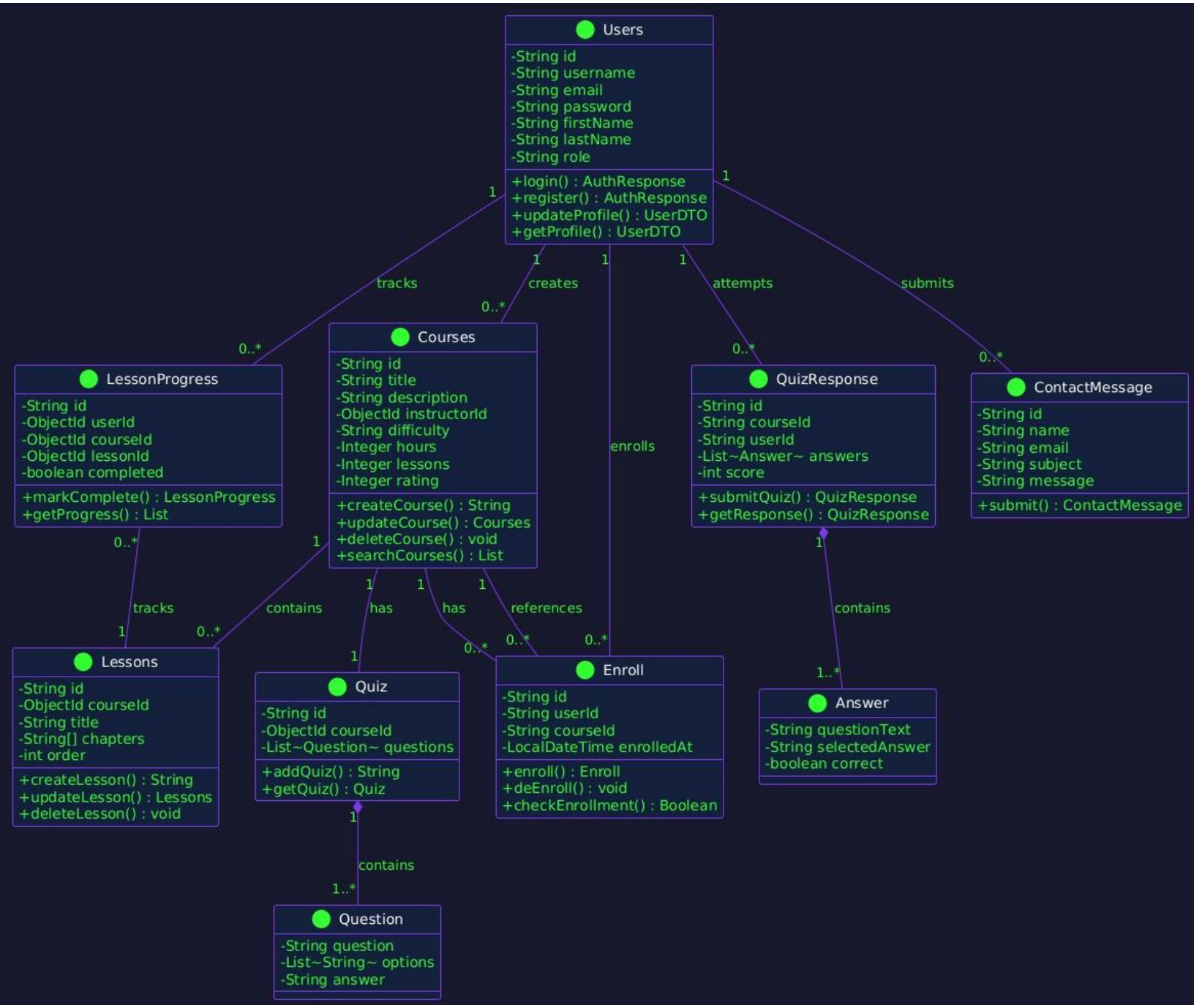
- User Registration & Authentication (UC-01, UC-02) — Role-based signup (Student / Instructor) with JWT token-based login, logout, and session management via localStorage.
- Course Creation & Content Upload (UC-03, UC-05) — Instructors create courses in a 3-step flow: course details → lessons with chapters → quiz questions with correct answers.
- Course Enrollment (UC-04) — Students browse and enroll in published courses with duplicate enrollment prevention (409 conflict handling).
- Learning Dashboard & Progress Tracking (UC-06) — Students complete lessons chapter by chapter with real-time progress bar, lesson completion tracking persisted in MongoDB, and a final quiz unlocked only after all lessons are done.
- Student Monitoring (UC-07) — Instructors view all enrolled students per course with per-student lesson completion percentages and completion badges.
- Course Search (UC-08) — Real-time keyword search across course titles and descriptions using MongoDB regex queries.
- User Profile Management (UC-09) — Authenticated users can view and update their name and password, with changes persisted to MongoDB and reflected immediately in the UI.
- Contact & Support (UC-10) — A contact form accessible without authentication that persists messages to MongoDB for follow-up.
- MVC + JWT Security — Full Spring Security filter chain with stateless JWT authentication, CORS configured for localhost:3000, and complete separation of Model, View, and Controller layers.

Models:

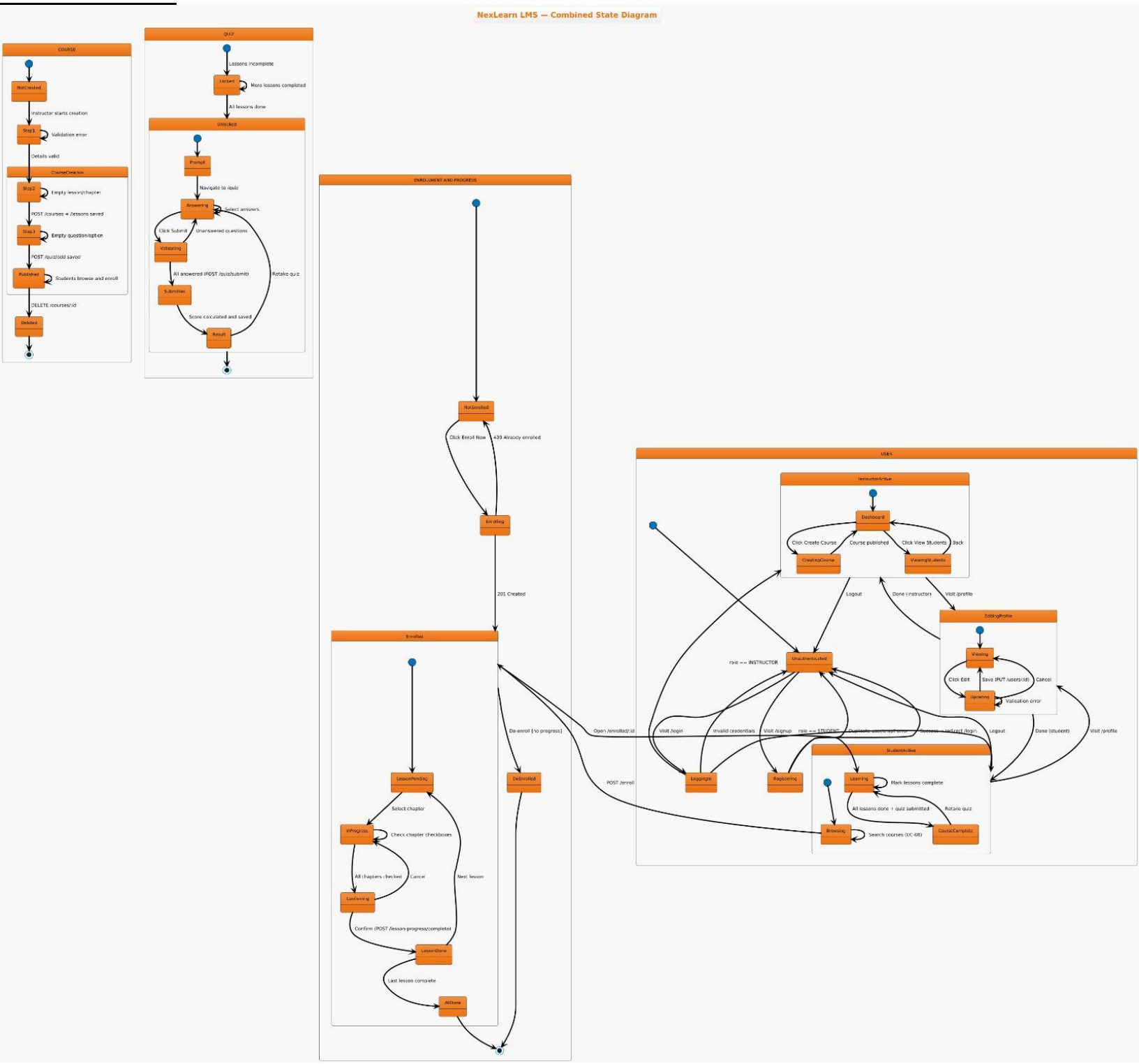
Use Case Diagram:



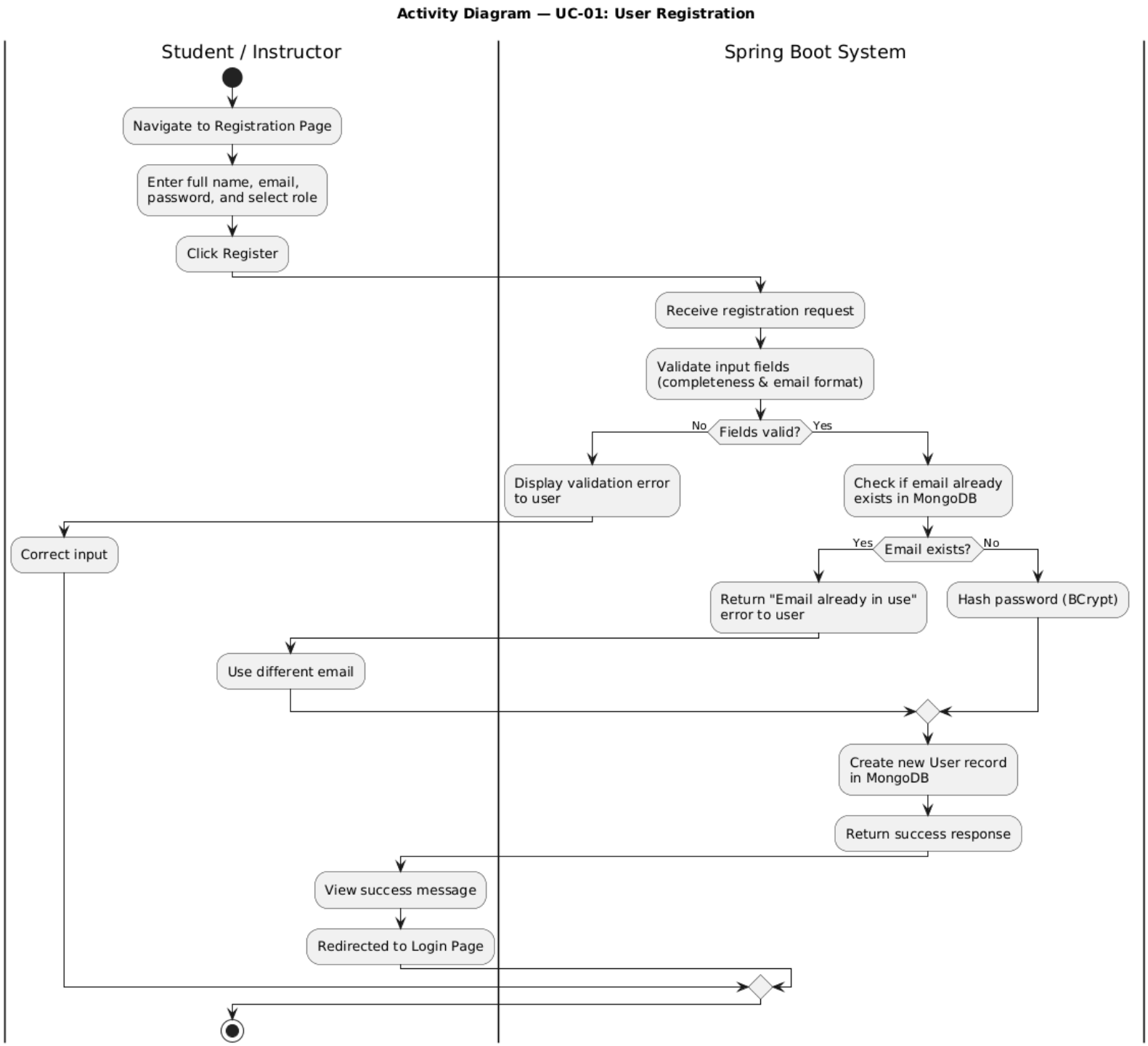
Class Diagram:



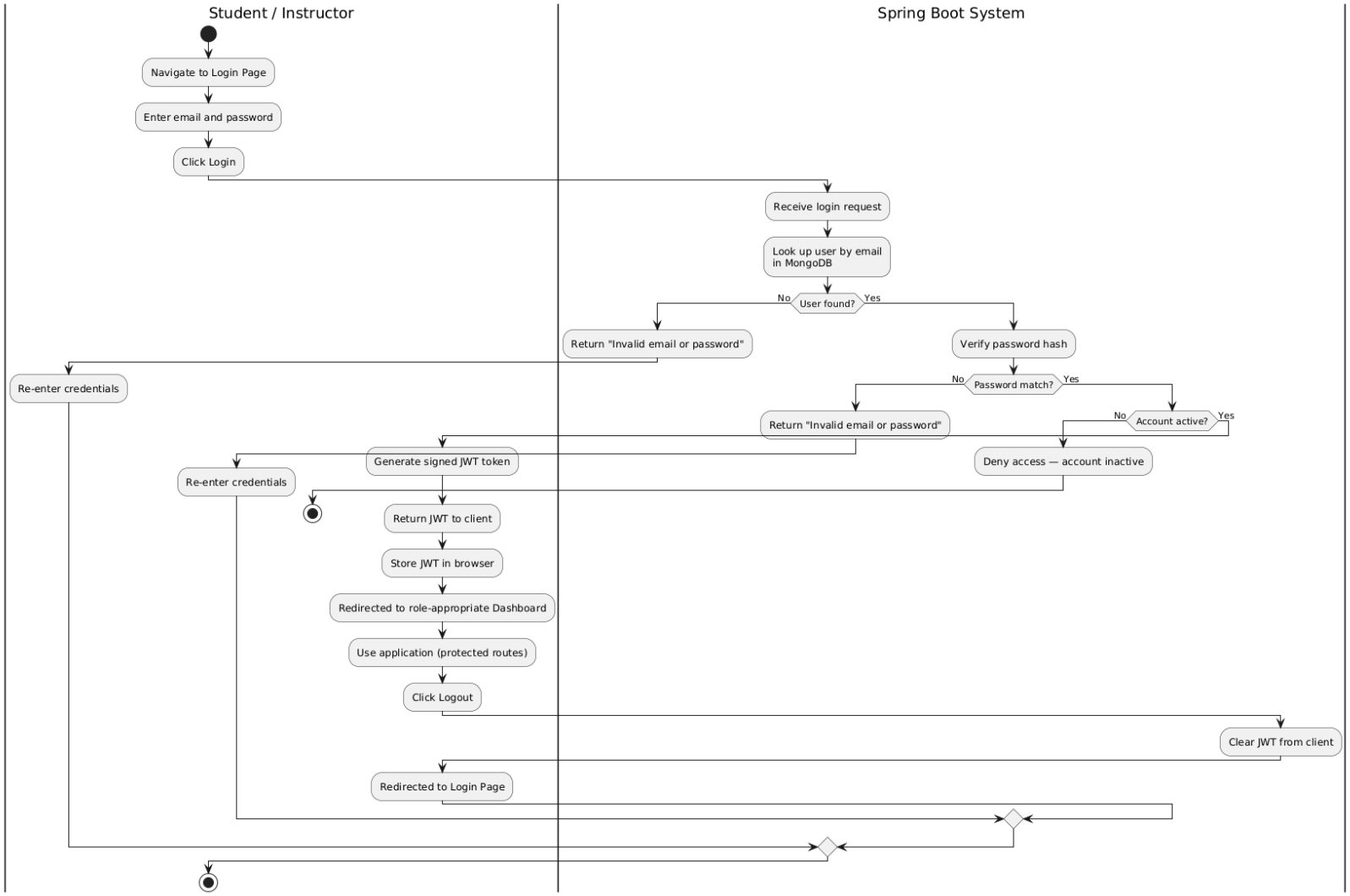
State Diagram:



Activity Diagrams:

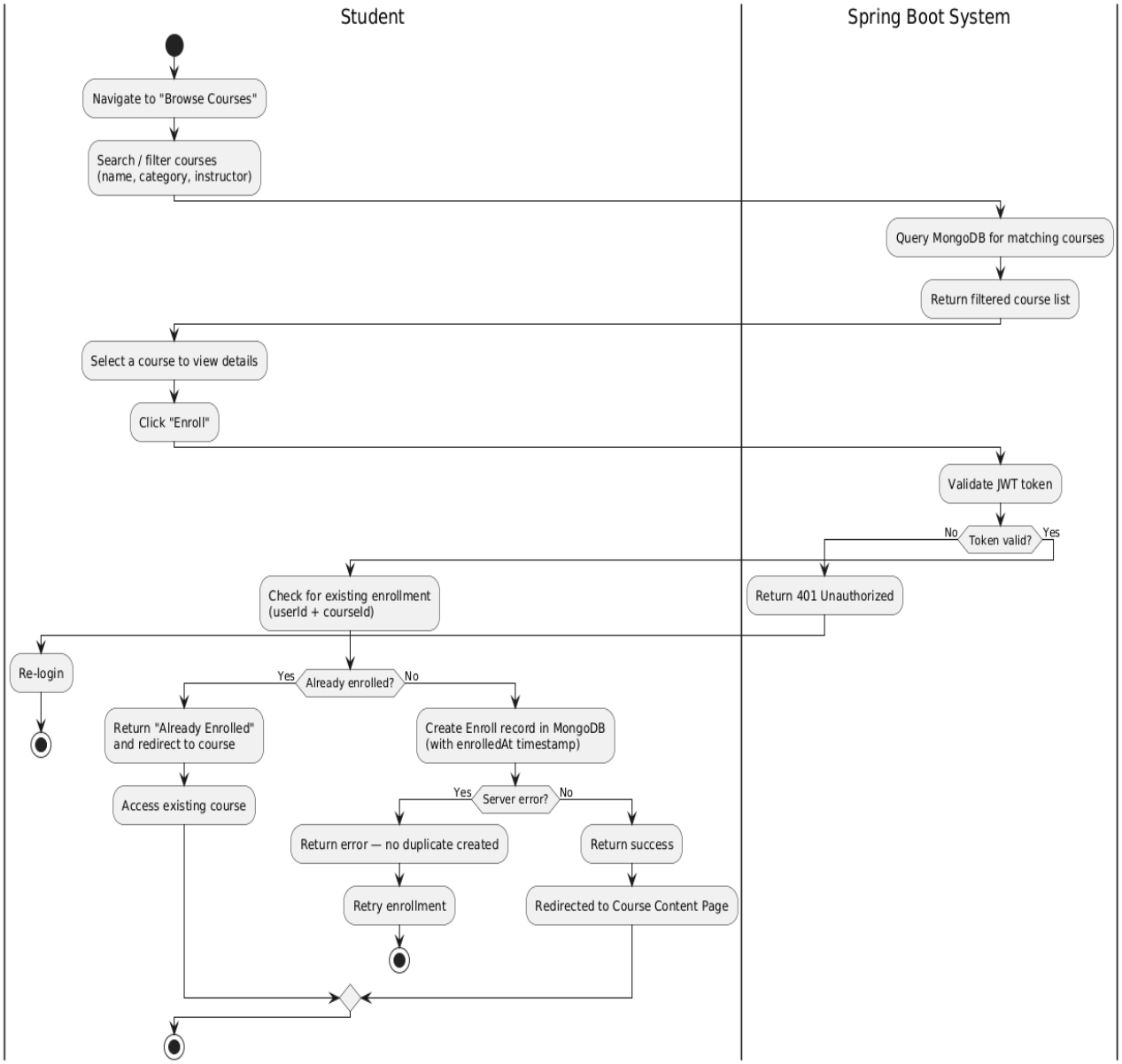


Activity Diagram — UC-02: User Authentication (Login/Logout)

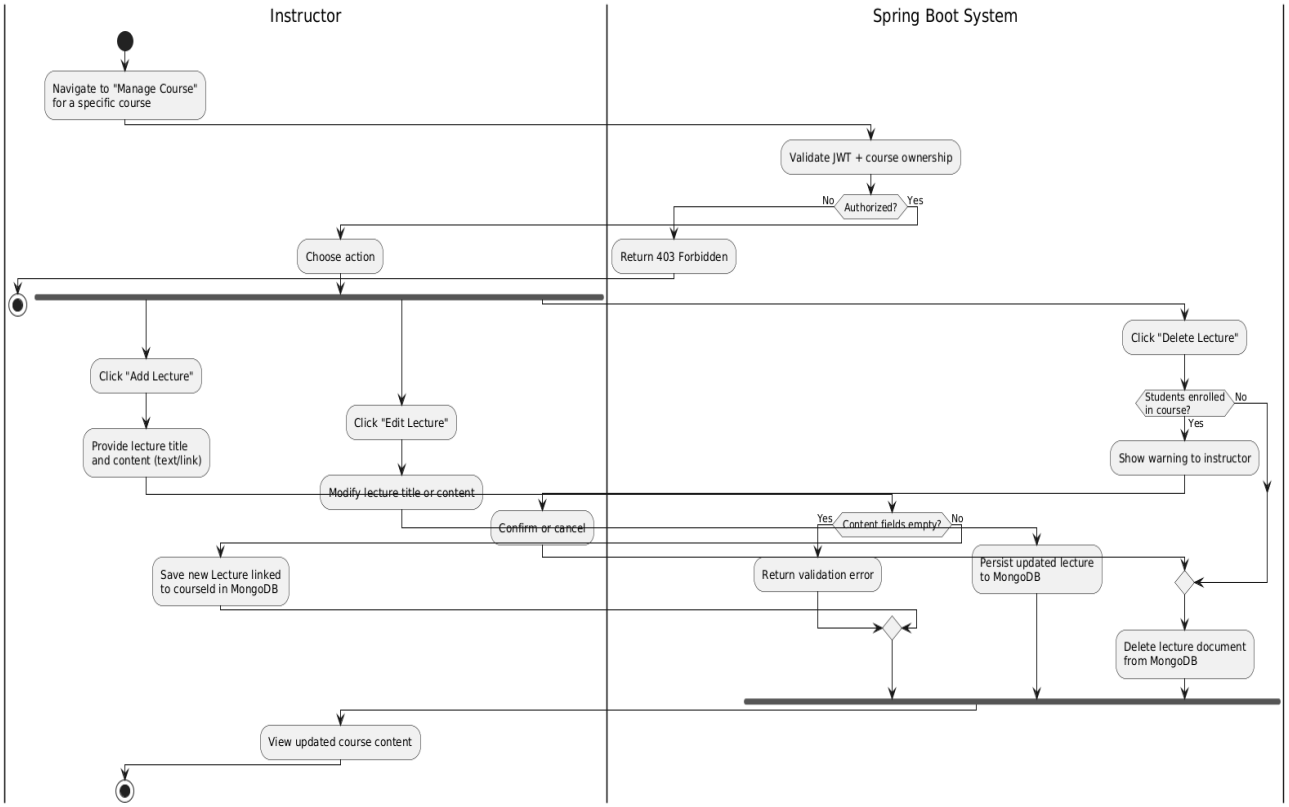


Activity Diagram — UC-03: Course Creation

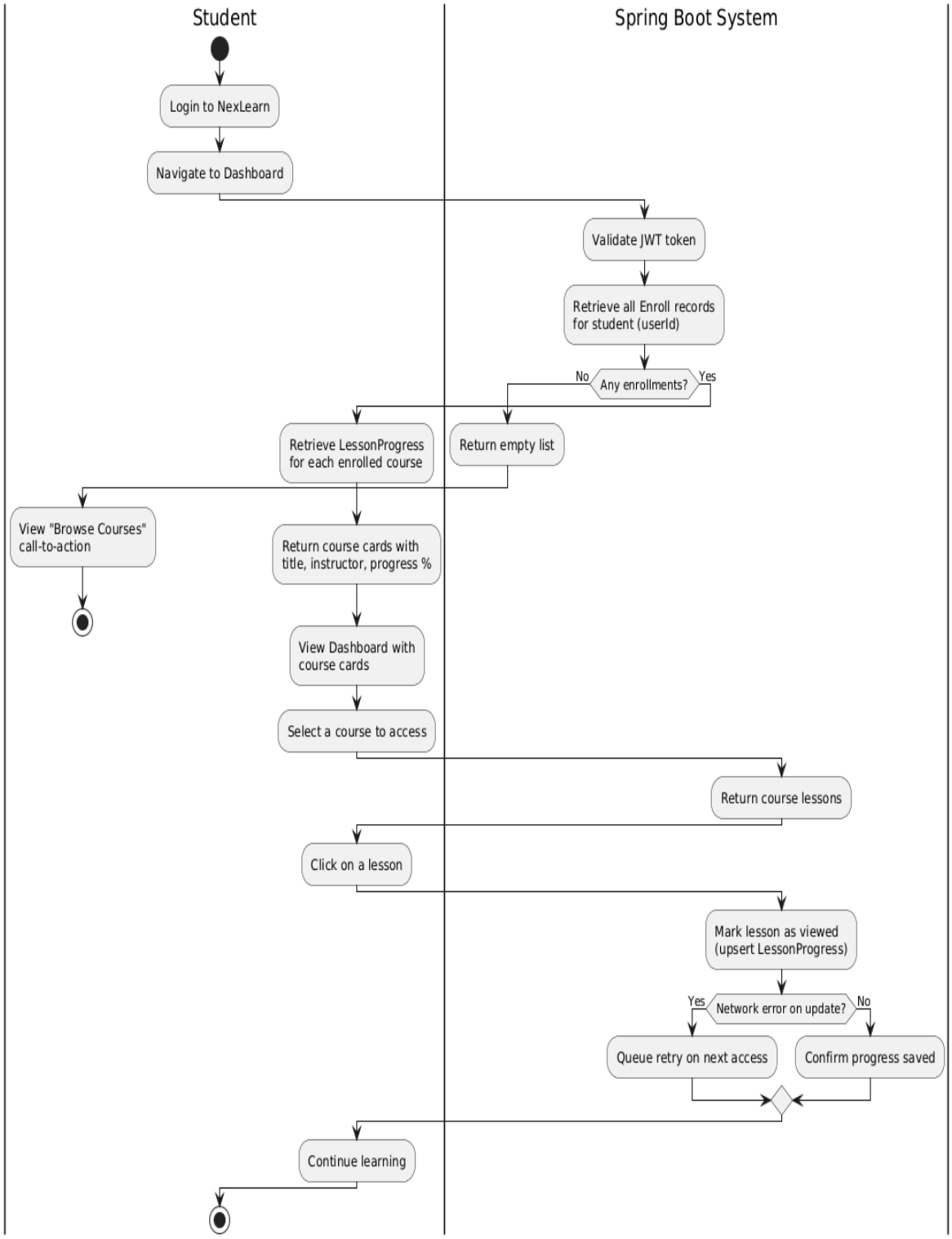
Activity Diagram — UC-04: Course Enrollment



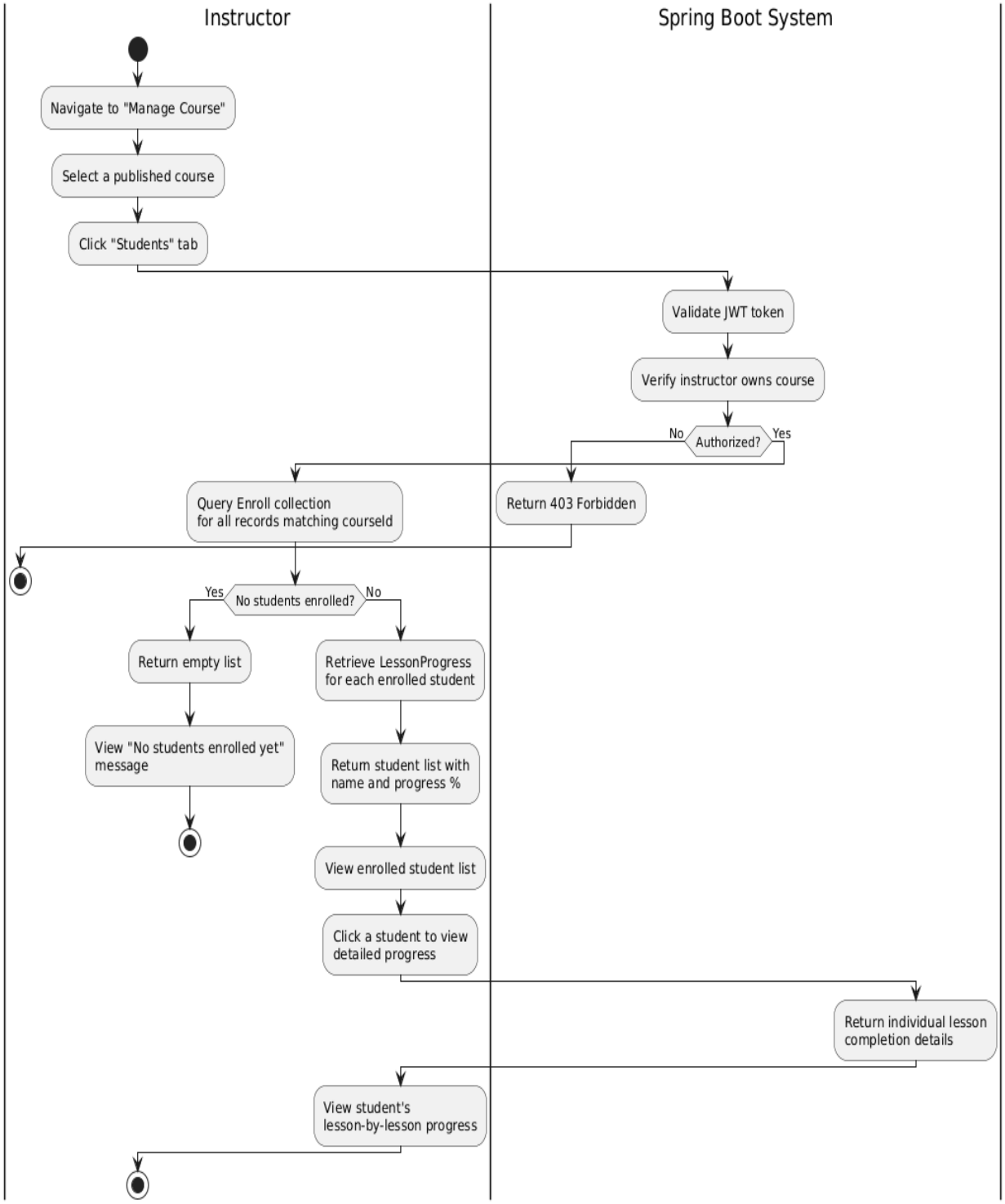
Activity Diagram — UC-05: Content Upload & Management



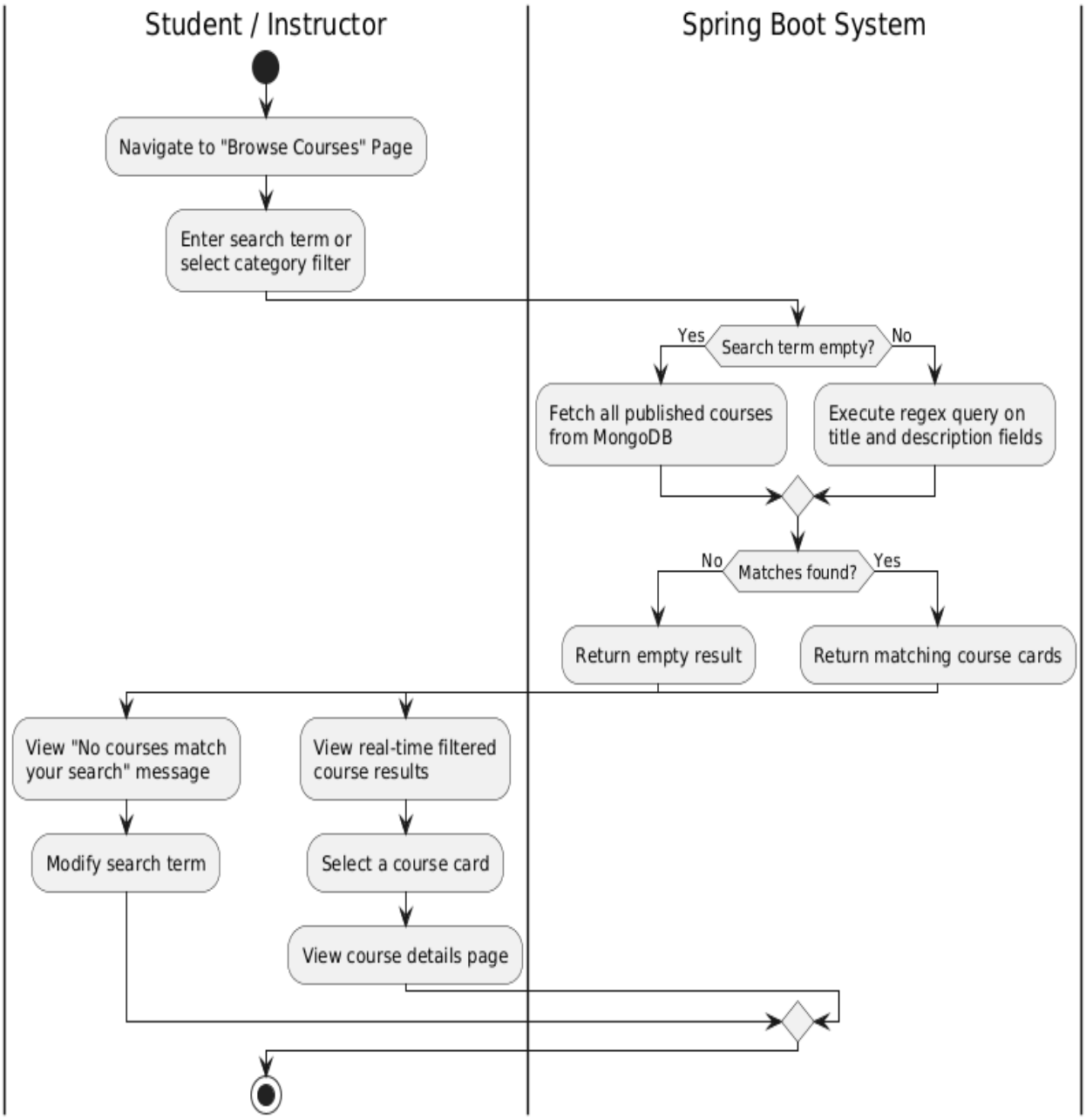
Activity Diagram — UC-06: Learning Dashboard & Progress Tracking



Activity Diagram — UC-07: Student Monitoring by Instructor

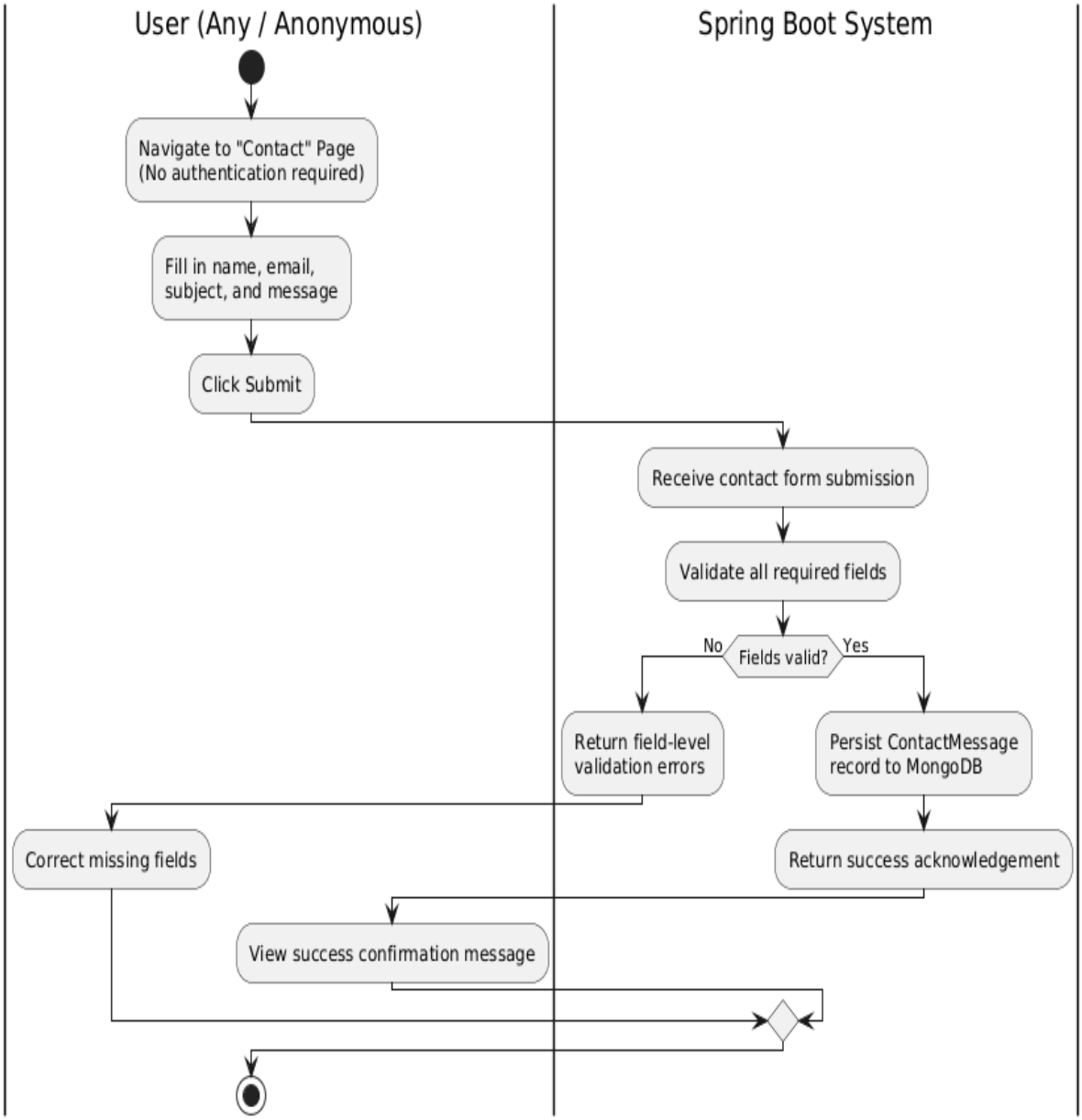


Activity Diagram — UC-08: Search Courses



Activity Diagram — UC-09: User Profile Management

Activity Diagram — UC-10: Contact & Support



Design Principles, and Design Patterns:

<u>MVC Architecture used?</u> Yes , it is used

<u>Design Principles</u>

SOLID Principles
S — Single Responsibility Principle
Every class has one job and one reason to change.
ClassSingle ResponsibilityJwtServiceOnly generates and validates JWT tokensDataSeederOnly seeds the database on startupContactControllerOnly handles contact form HTTP requestsEnrollServiceOnly handles enrollment business logicLessonProgressServiceOnly handles lesson completion logic

O — Open/Closed Principle
Classes are open for extension, closed for modification.
UC-10 (Contact & Support) was added as a completely new set of classes — ContactController, ContactService, ContactRepository, ContactMessage — without touching a single existing class. Same for DataSeeder — added without modifying any existing Spring component.

L — Liskov Substitution Principle
All repository interfaces extend MongoRepository. Any repository can be substituted where MongoRepository is expected without breaking anything.
java// CourseRepository IS-A MongoRepository
// EnrollRepository IS-A MongoRepository
// All behave identically from the caller's perspective
public interface CourseRepository extends MongoRepository<Courses, String> {}
public interface EnrollRepository extends MongoRepository<Enroll, String> {}

I — Interface Segregation Principle
Each repository interface is small and specific — it only declares the methods needed for that entity. No fat interfaces.
java// UserRepository only has what Users needs
public interface UserRepository extends MongoRepository<Users, String> {
    Optional<Users> findByUsername(String username);
    Optional<Users> findByEmail(String email);
}
// Not burdened with course or lesson methods

D — Dependency Inversion Principle
High-level modules depend on abstractions, not concrete classes. Spring's @Autowired injects implementations at runtime.
java// CoursesController depends on CourseService (abstraction)
// NOT on CourseRepository directly
@Autowired
private CourseService courseService;

// CourseService depends on CourseRepository (abstraction)
@Autowired
private CourseRepository courseRepository;

GRASP Principles
1. Information Expert
Responsibility is assigned to the class that has the information needed to fulfill it.
ResponsibilityAssigned ToWhyCalculate quiz scoreQuizServiceHas access to Quiz questions and submitted answersCheck duplicate enrollmentEnrollController + EnrollServiceHas access to EnrollRepositoryMark lesson completeLessonProgressServiceHas all progress dataSeed sample dataDataSeederHas access to all repositories

2. Creator
A class creates objects it aggregates or closely uses.
CreatorCreatesAuthControllerCreates Users and UserDTO objectsEnrollControllerCreates Enroll objectsDataSeederCreates Users, Courses, Lessons, Quiz, Question objectsQuizServiceCreates and updates QuizResponse objects

3. Controller
A Controller class handles system events and delegates to services. In NexLearn every @RestController acts as a GRASP Controller — it receives the HTTP request and immediately delegates to the Service layer without doing business logic itself.
java// AuthController — thin controller, delegates to service
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody AuthRequest request) {
    // Only orchestrates — no business logic here
    Authentication auth = authenticationManager.authenticate(...);
    String token = jwtService.generateToken(userDetails);
    return ResponseEntity.ok(new AuthResponse(token, userDTO));
}

4. Low Coupling

Classes are loosely coupled through Spring dependency injection. The frontend (Next.js) is completely decoupled from the backend — it only communicates via REST API calls. No frontend file imports any Java class.

5. High Cohesion
Each class is focused on one related set of responsibilities.
ClassCohesionJwtServiceAll methods relate to JWT onlyLessonProgressServiceAll methods relate to progress onlyQuizServiceAll methods relate to quiz + scoringNavbar.tsxOnly handles navigation UI

6. Polymorphism
Spring Security uses polymorphism through UserDetailsService. UserDetailsServiceImpl implements UserDetailsService — Spring calls loadUserByUsername() without knowing the concrete implementation.
javapublic class UserDetailsServiceImpl implements UserDetailsService {

```
    @Override
    public UserDetails loadUserByUsername(String username) { ... }
}
```

7. Pure Fabrication
JwtService, DataSeeder, and UserDetailsServiceImpl are Pure Fabrication classes — they don't represent real-world domain objects but are created purely to achieve low coupling and high cohesion in the system.

8. Indirection

Indirection assigns responsibility to an intermediate object to mediate between two components so they are not directly coupled.
Where in NexLearn:

The Service layer acts as the indirection layer between Controllers and Repositories. Controllers never talk to repositories directly — they always go through a Service.

CoursesController → CourseService → CourseRepository → MongoDB
EnrollController → EnrollService → EnrollRepository → MongoDB
QuizController → QuizService → QuizRepository → MongoDB

JwtAuthenticationFilter is also an indirection layer — it sits between every HTTP request and every Controller, mediating authentication without the Controllers knowing anything about how JWT works.

HTTP Request → JwtAuthenticationFilter → Controller

The REST API itself is the biggest indirection in the system — it sits between the Next.js frontend and the Spring Boot backend so neither side is coupled to the other's implementation.

Next.js Frontend → REST API (indirection) → Spring Boot Backend

Why it matters: If you swapped MongoDB for PostgreSQL, only the Repository layer changes. The Controllers and the frontend are completely unaffected because of the Service indirection layer in between.

9. Protected Variations
Protected Variations protects elements from the variations in other elements by wrapping the unstable or likely-to-change parts behind a stable interface.

Where in NexLearn:

(i) Repository interfaces protect against database changes All repositories extend MongoRepository which is a stable Spring Data interface. If the database changes from MongoDB to any other Spring Data supported database, the Controllers and Services are fully protected — only the repository implementations change.
java
```
// Stable interface — everything above this is protected
public interface CourseRepository extends MongoRepository<Courses, String> {
    List<Courses> findByInstructorId(ObjectId instructorId);
}
```

(ii) PasswordEncoder bean protects against hashing algorithm changes The PasswordEncoder is injected as a bean in SecurityConfig. Switching from NoOpPasswordEncoder to BCryptPasswordEncoder requires changing exactly one line. Every other class (AuthController, UserDetailsServiceImpl) is protected from this variation.
java
```
@Bean
public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
    // change to BCrypt here — nothing else in the app changes
}
```

(iii) UserDTO protects against Users model changes UserDTO is a stable transfer object that shields the frontend and controllers from internal changes to the Users model. If a new field is added to Users (like profilePicture), the API response stays the same until UserDTO is explicitly updated.

java
// Users model can change internally
// Frontend always receives this stable shape
UserDTO(id, username, email, firstName, lastName, role)

(iv) JWT filter protects Controllers from authentication logic changes All Controllers are protected from any change in how authentication works. Whether JWT expiry changes, the secret rotates, or the algorithm switches from HS256 to RS256 — no Controller needs to change because JwtAuthenticationFilter and JwtService absorb all that variation.

Design Patterns

CREATIONAL PATTERNS

1. Singleton Pattern
What it is: Only one instance of a class exists throughout the application lifecycle.
Where in NexLearn: Every Spring @Service, @Repository, @Component, and @RestController bean is a Singleton by default — Spring creates exactly one instance and reuses it for every request.

```
java@Service  // Spring creates ONE instance of CourseService
public class CourseService {
   @Autowired private CourseRepository courseRepository;
   // This same instance handles ALL course-related requests
}

@Component  // ONE instance of DataSeeder exists
public class DataSeeder implements CommandLineRunner {
   // Runs exactly once on startup
}
```

Impact: Memory efficient — JwtService, QuizService, LessonProgressService etc. are all instantiated once and shared. No redundant object creation per request.

2. Builder Pattern
What it is: Constructs complex objects step by step.
Where in NexLearn: JwtService uses the JJWT library's builder pattern to construct JWT tokens step by step.

```
javareturn Jwts.builder()              // Step 1: start builder
   .setClaims(extra)             // Step 2: set claims
   .setSubject(username)         // Step 3: set subject
   .setIssuedAt(new Date(...))    // Step 4: set issue time
   .setExpiration(new Date(...))  // Step 5: set expiry
```

```
    .signWith(getSignInKey(), ...) // Step 6: sign
    .compact();                    // Step 7: build final token
```

Also: Spring Security's UsernamePasswordAuthenticationToken and ResponseEntity use the builder pattern internally.
Impact: JWT construction is clean and readable — each step is explicit and the order is enforced by the builder chain.

3. Factory Method Pattern
What it is: Delegates object creation to a method rather than calling new directly.
Where in NexLearn: AuthController.register() acts as a factory for Users objects. The controller decides what to create based on the incoming RegisterRequest.

```
java@PostMapping("/register")
public ResponseEntity<?> register(@RequestBody RegisterRequest request) {
    // Factory method — creates Users based on request data
    Users user = new Users(
        request.getUsername(),
        request.getEmail(),
        request.getPassword(),
        request.getRole(),       // Determines STUDENT or INSTRUCTOR
        request.getFirstName(),
        request.getLastName()
    );
    userRepository.save(user);
    ...
}
```

Also: DataSeeder.run() is a factory that creates and wires together Users, Courses, Lessons, Question, and Quiz objects on startup.
Impact: Object creation logic is centralised — if the Users constructor changes, only the factory method needs updating.

STRUCTURAL PATTERNS

4. Repository Pattern
What it is: Abstracts the data access layer behind an interface so the rest of the application doesn't know how data is stored or queried.
Where in NexLearn: Every MongoRepository interface is a Repository pattern implementation.

```
javapublic interface CourseRepository extends MongoRepository<Courses, String> {
    @Query("{instructor_id: ?0}")
    List<Courses> findByInstructorId(ObjectId instructorId);
```

```
    @Query("{ $or: [ { title: { $regex: ?0, $options: 'i' } } ] }")
    List<Courses> searchCourses(String keyword);
}
```

All repositories:
RepositoryResponsibilityUserRepositoryFind by username, emailCourseRepositoryFind by instructor, search by keywordEnrollRepositoryFind by userId + courseId, delete by bothLessonRepositoryFind by courseId ObjectIdLessonProgressRepositoryFind by userId + courseId + lessonIdQuizRepositoryFind by courseIdQuizResponseRepositoryFind by courseId + userIdContactMessageRepositorySave contact messages
Impact: If MongoDB is replaced with PostgreSQL, only the repository implementations change — CourseService, CoursesController, and all other layers remain untouched.

5. Facade Pattern
What it is: Provides a simplified interface to a complex subsystem.
Where in NexLearn: Every @Service class is a Facade. CourseService hides the complexity of MongoDB queries, ObjectId conversions, and data access behind simple method calls.

```
java// Controller sees a simple facade — one method call
public String createCourse(Courses course) {
    return courseRepository.save(course).getId();
}

// QuizService facade hides complex scoring logic
public QuizResponse submitQuiz(QuizResponse response) {
    Quiz quiz = quizRepository.findByCourseId(new ObjectId(response.getCourseId()));
    // Complex scoring logic hidden from controller
    int score = 0;
    for (int i = 0; i < answers.size(); i++) {
        boolean correct = q.getAnswer().equals(a.getSelectedAnswer());
        a.setCorrect(correct);
        if (correct) score++;
    }
    response.setScore(score);
    return quizResponseRepository.save(response);
}
```

Also: The REST API itself is a Facade — the Next.js frontend calls simple endpoints like POST /enroll/:userId/:courseId without knowing anything about MongoDB, Spring Security, or JWT internals.
Impact: Controllers stay thin and readable. The entire scoring algorithm lives in QuizService — if it changes, the controller doesn't need touching.

6. Proxy Pattern
What it is: An object that controls access to another object.

Where in NexLearn: JwtAuthenticationFilter is a security proxy. It intercepts every HTTP request before it reaches any controller and validates the JWT token. The controller only runs if the proxy allows it through.

```
java@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(request, response, filterChain) {
        // PROXY: intercepts every request
        String jwt = authHeader.substring(7);
        String username = jwtService.extractUsername(jwt);

        if (jwtService.isTokenValid(jwt, userDetails)) {
            // Set authentication context — allow through
            SecurityContextHolder.getContext().setAuthentication(authToken);
        }
        // Pass to actual controller only if valid
        filterChain.doFilter(request, response);
    }
}
```

Impact: No controller needs to check authentication itself — the proxy handles it transparently for the entire application.

BEHAVIORAL PATTERNS

7. Template Method Pattern
What it is: Defines the skeleton of an algorithm in a base class and lets subclasses override specific steps.
Where in NexLearn: JwtAuthenticationFilter extends Spring's OncePerRequestFilter. The parent class defines the template (run once per request, handle exceptions, call filter chain). The child class overrides doFilterInternal() to implement NexLearn-specific JWT logic.

```
java// Parent defines the template
public abstract class OncePerRequestFilter extends GenericFilterBean {
    public final void doFilter(request, response, chain) {
        // Template: ensure runs once per request
        doFilterInternal(request, response, chain); // ← hook method
    }
    protected abstract void doFilterInternal(...); // ← override this
}

// NexLearn overrides the hook
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(...) {
        // NexLearn specific: extract and validate JWT
    }
}
```

Impact: Spring handles all the filter chain boilerplate. NexLearn only writes the JWT-specific logic.

8. Strategy Pattern
What it is: Defines a family of algorithms and makes them interchangeable.
Where in NexLearn: Spring Security uses the Strategy pattern for password encoding. SecurityConfig injects a PasswordEncoder strategy — currently NoOpPasswordEncoder for

the demo, but swappable to BCryptPasswordEncoder without changing any other class.
java@Bean
```
public PasswordEncoder passwordEncoder() {
    // Strategy: swap this one line to change encoding for entire app
    return NoOpPasswordEncoder.getInstance();
    // Can be replaced with: return new BCryptPasswordEncoder();
}
```
Also: AuthenticationManager uses the Strategy pattern — different authentication strategies (JWT, session-based, OAuth) can be plugged in through AuthenticationConfiguration.
Impact: Switching from plain text passwords to BCrypt only requires changing one line in SecurityConfig — zero changes to AuthController, UserService, or UserDetailsServiceImpl.

9. Observer Pattern
What it is: When one object changes state, all its dependents are notified automatically.
Where in NexLearn: DataSeeder implements CommandLineRunner — Spring Boot fires the run() method automatically when the application context finishes loading. DataSeeder is an observer of the Spring application lifecycle.
java@Component
```
public class DataSeeder implements CommandLineRunner {
    @Override
    public void run(String... args) {
        // Spring automatically calls this when app starts
        // DataSeeder observes ApplicationStartedEvent implicitly
        if (userRepository.count() > 0) return;
        // Seed data...
    }
}
```
Also: On the frontend, React's useEffect hook is the Observer pattern — components observe state changes and re-render automatically when completedLessons, courses, or user state changes.
tsxuseEffect(() => {
```
    // Observes: id changes → re-fetch course and progress
    fetchProgress(user, token);
}, [id]);
```
Impact: The seeding logic runs automatically without any manual call. Frontend UI updates reactively when lesson progress or enrollment state changes.

10. Command Pattern
What it is: Encapsulates a request as an object.
Where in NexLearn: Every REST API request is a Command. RegisterRequest, AuthRequest, and UpdateProfileRequest DTOs are Command objects — they encapsulate all data needed to execute an operation.

```
java// RegisterRequest is a Command object
public class RegisterRequest {
   private String username;
   private String email;
   private String password;
   private String firstName;
   private String lastName;
   private String role;
   // Encapsulates everything needed for registration
}

// AuthRequest is a Command object
public class AuthRequest {
   private String username;
   private String password;
}
```

Impact: Controllers receive clean command objects — no raw parameter parsing. DTOs protect the domain model from direct exposure.

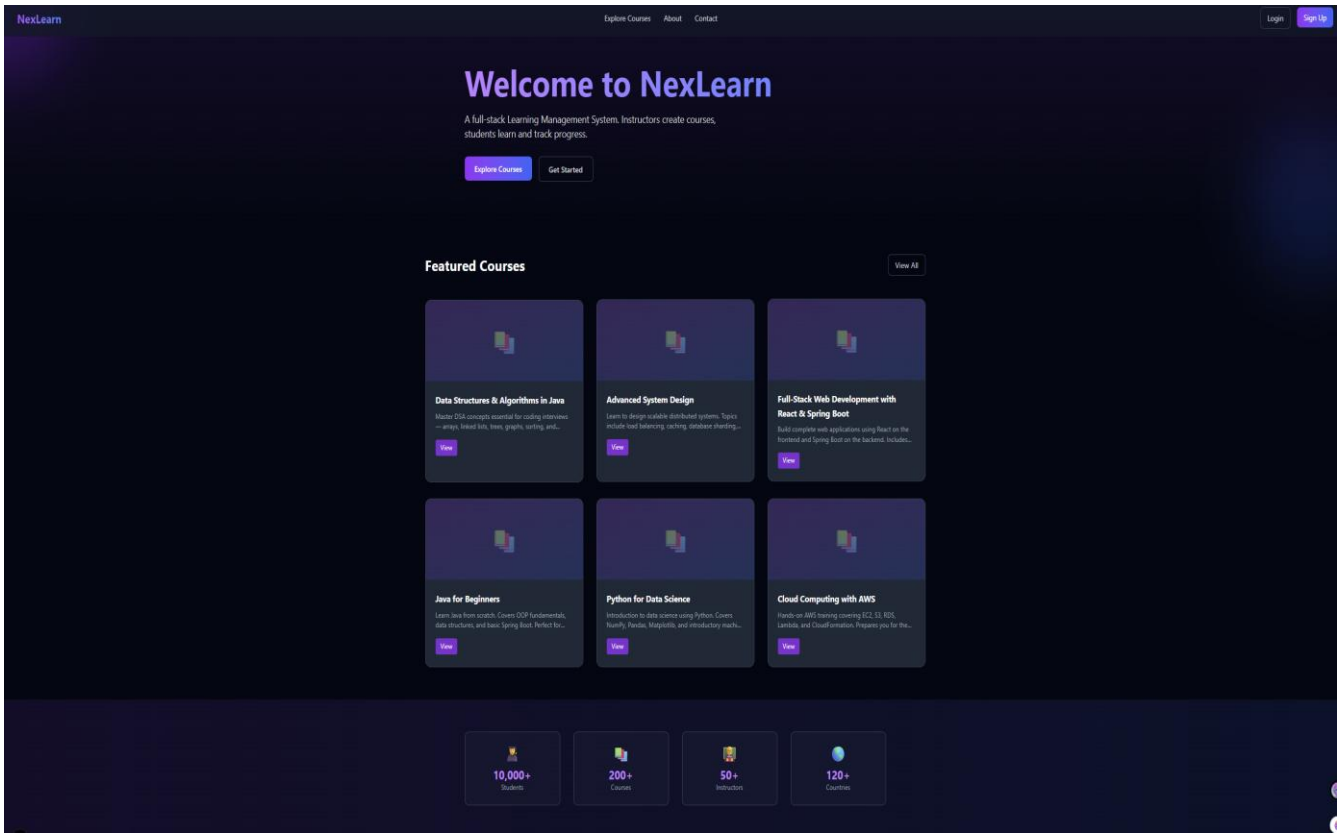
Github link to the Codebase: (repository should be public)

https://github.com/Kathitjoshi/NexLearn

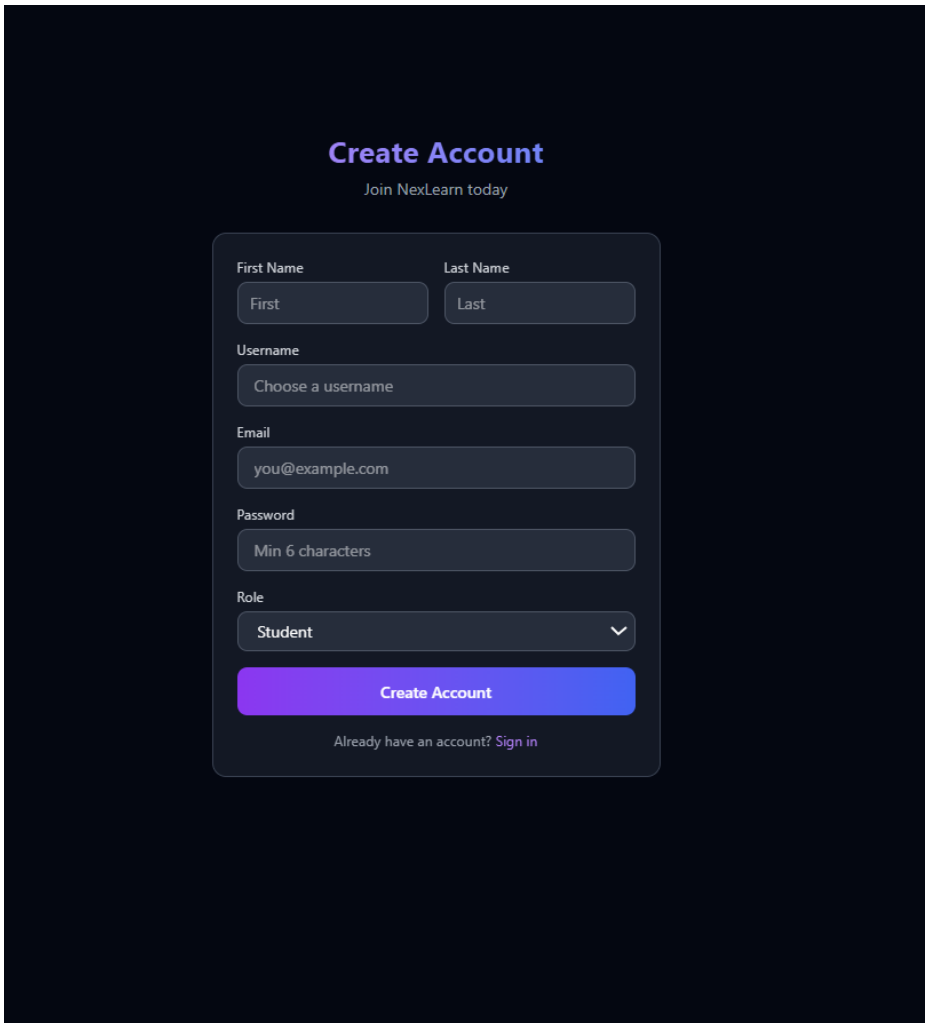
Screenshots

UI:

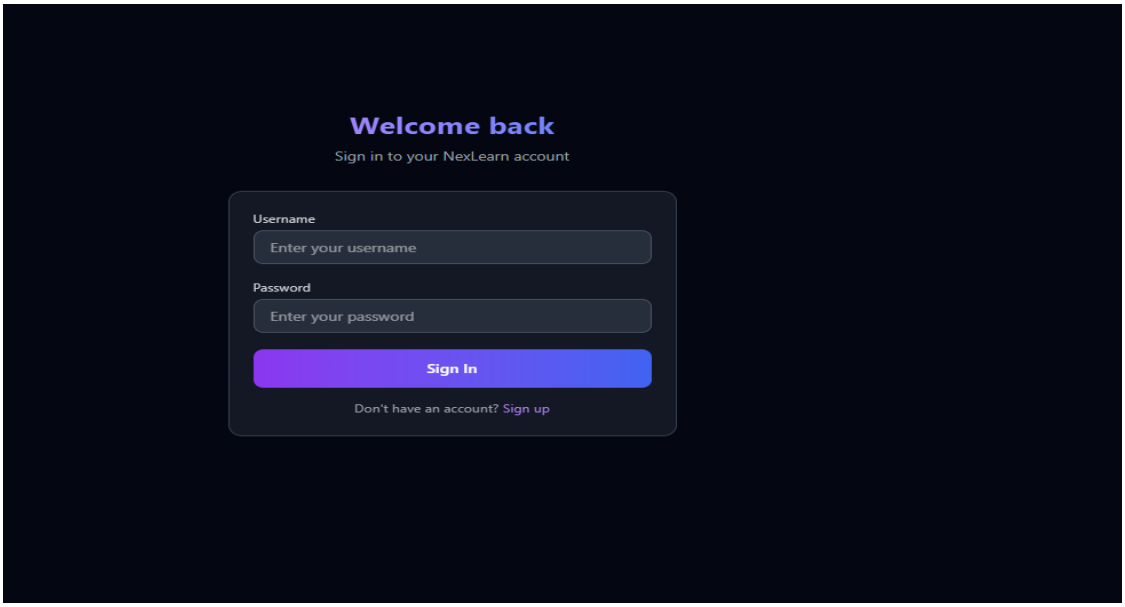
1) Home Page



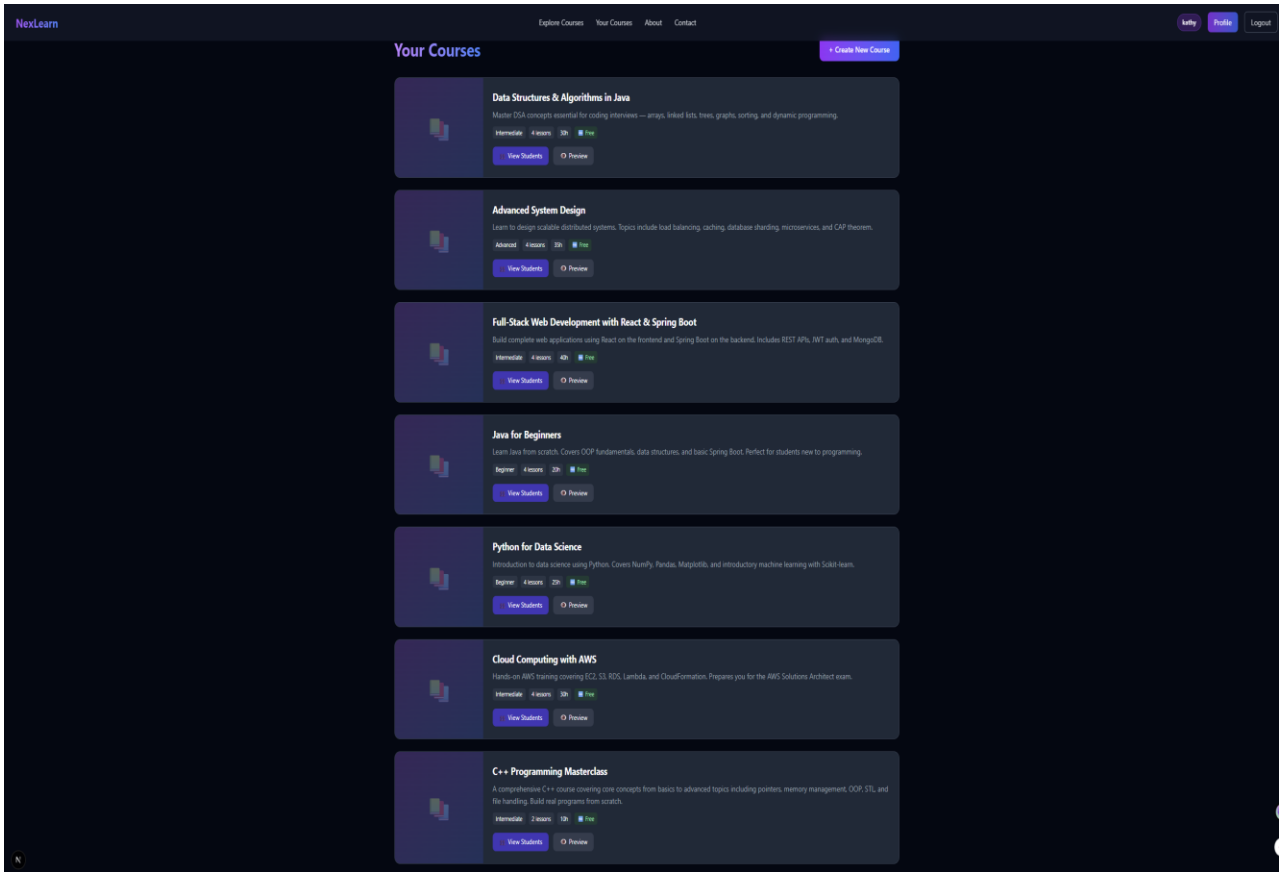
2) Sign Up page



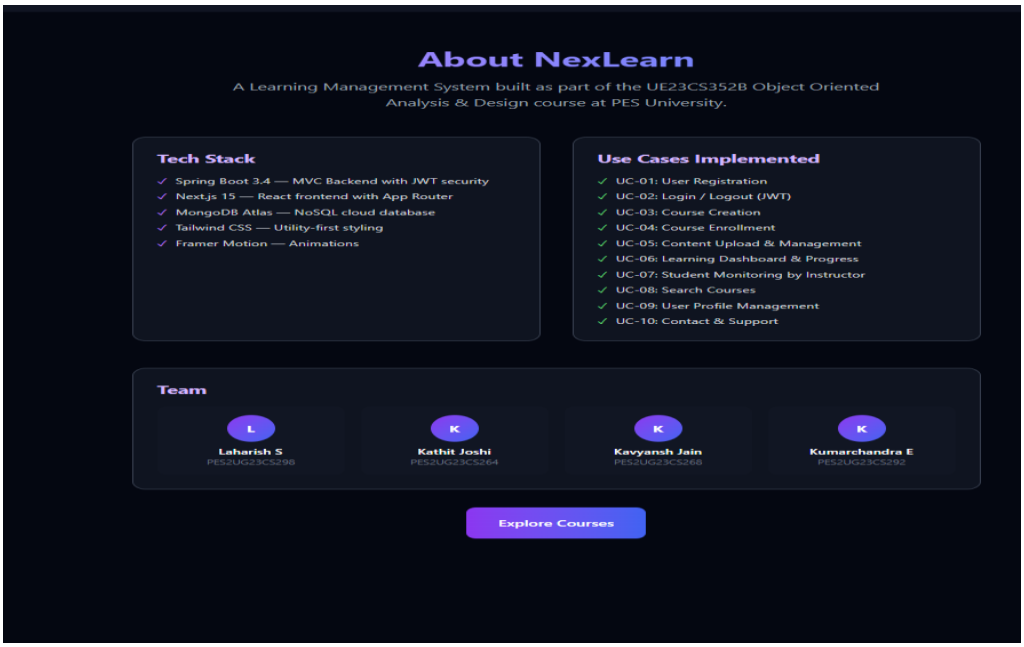
3) Login Page



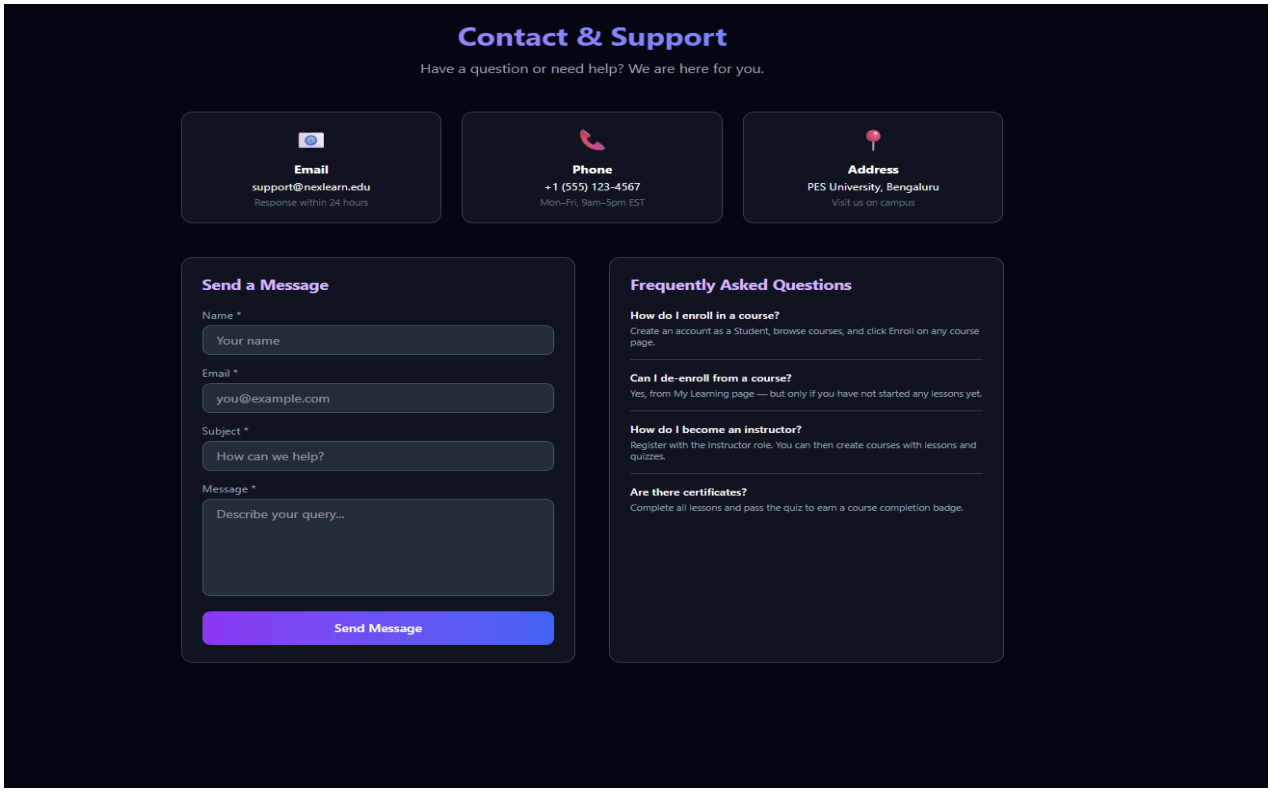
4) Courses Page from instructor side



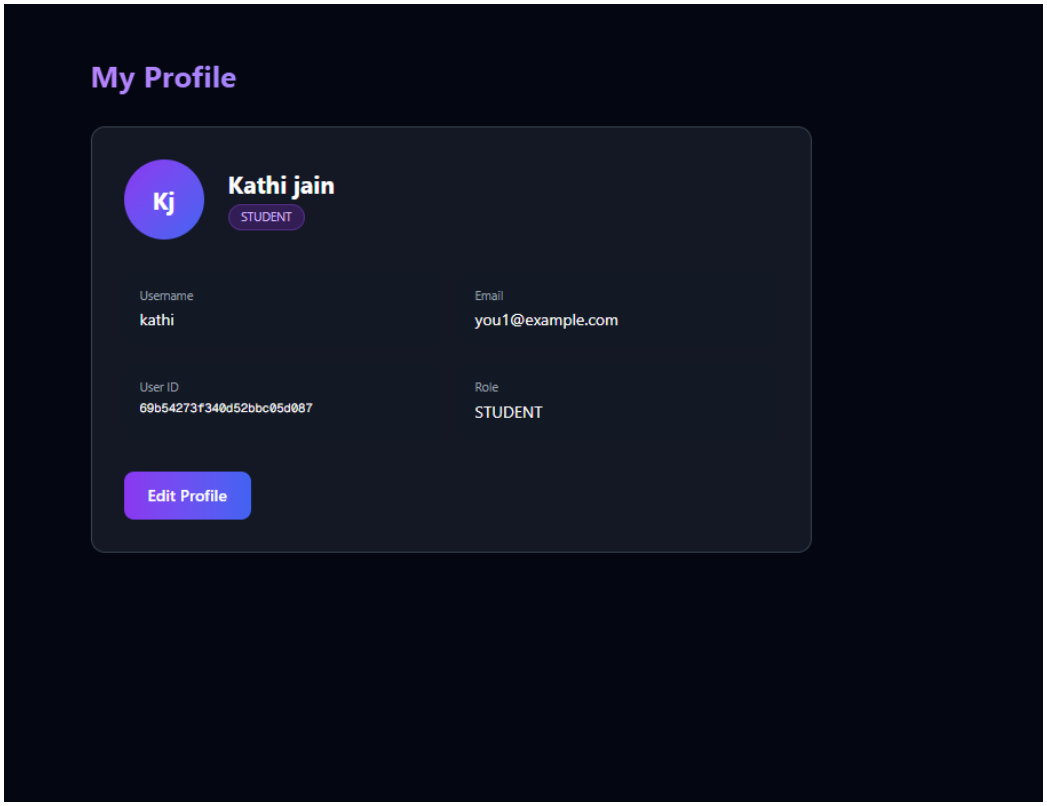
5) About Page (Same for both)

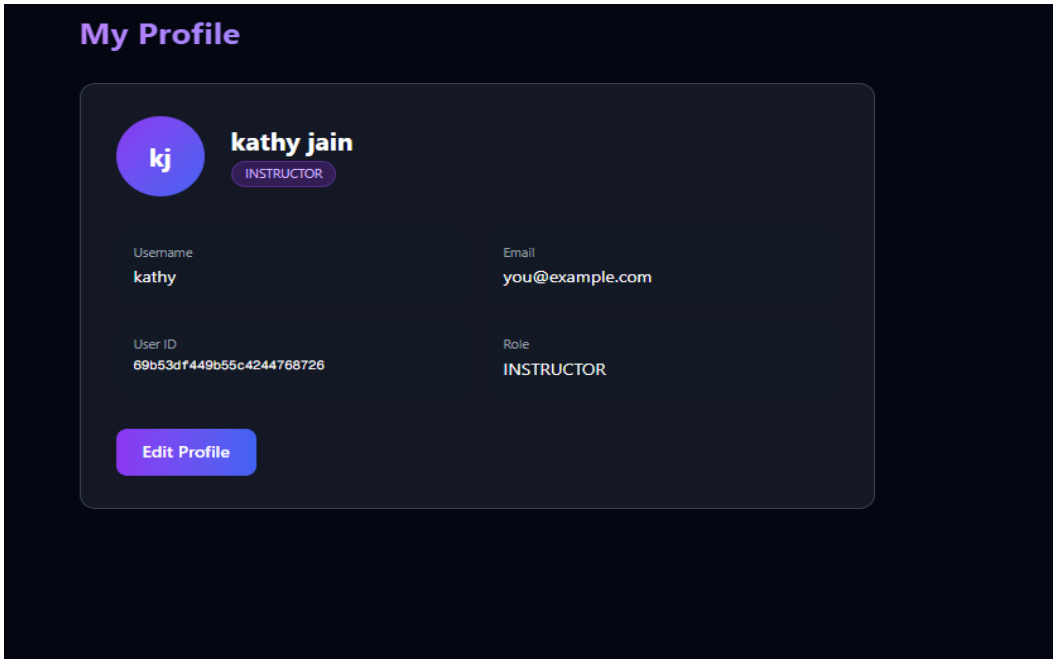


6) Contact and Support Page (Same for both)

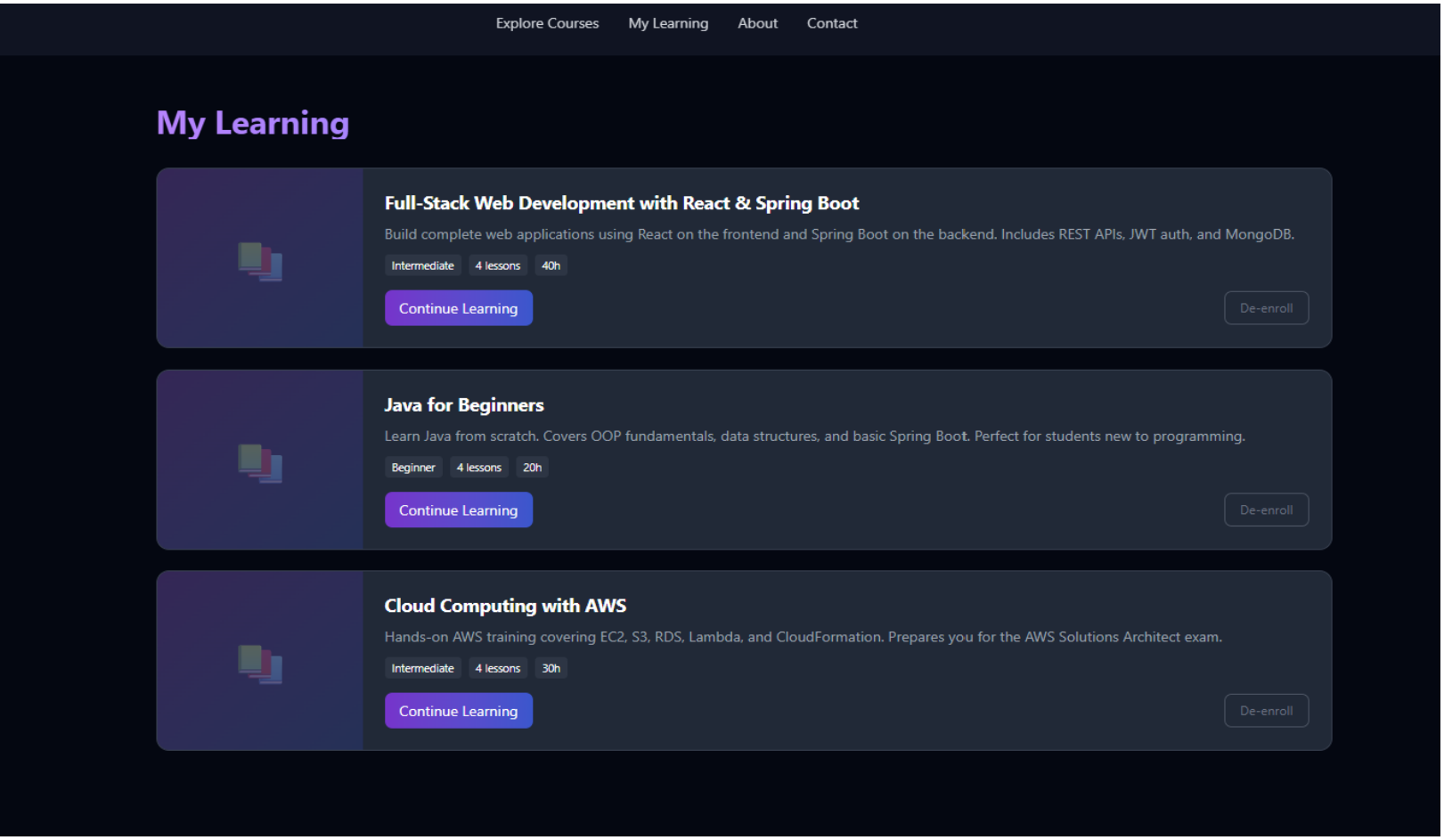


7) Profile Page





8) My Learning Page from Student side



Individual contributions of the team members:

Name	**Module worked on**
Kathit Joshi	UC01 TO UC04 + SOLID principles check + Database
Laharish S	UC 05 to UC06 + GRASP principles check
Kumarchandra Edupuganti	UC07 to UC08 + Design Pattern check
Kavyansh Jain	UC09 to UC10 + MVC Check